Questions on Human Activity Recognition

Lab Assignment Report

Raj Shekhar Singh (24116079) Rith Rajak (24116085)

1 Question 1 – Data Collection and Feature Extraction

1.1 Part A – Recording Sensor Data

For this question, I used the **Physics Toolbox Sensor Suite** app on my Android phone to record accelerometer and gyroscope data. The phone was held in the hand during all activities. The activities I recorded were:

- Walking
- Sitting
- Standing
- Jumping
- Back push (pushing hand backward)
- Front push (pushing hand forward)

Each activity was performed for approximately **30 seconds**. The sampling rate was set to **100 Hz** (100 samples per second). The data was exported as CSV files with columns: `time, acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z`.

Note: For walking, I kept the phone in my shirt pocket. For jumping and push activities, I held the phone firmly in my hand so the sensor data is more clean.

1.2 Part B – Processing in Python

I processed all the data using Python (Google Colab). Below is the code I used:

Code (Importing libraries and loading data):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
from scipy.signal import butter, filtfilt

df_walking = pd.read_csv('walking.csv')
```

```
df_sitting = pd.read_csv('sitting.csv')
df_standing = pd.read_csv('standing.csv')
df_jumping = pd.read_csv('jumping.csv')
df_back = pd.read_csv('back_push.csv')
df_front = pd.read_csv('front_push.csv')

activities = {
    'Walking': df_walking,
    'Sitting': df_sitting,
    'Standing': df_standing,
    'Jumping': df_jumping,
    'Back Push': df_back,
    'Front Push': df_front
}
```

Code (Low-pass filter to remove noise):

```
def butter_lowpass_filter(data, cutoff=20, fs=100, order=4):
    nyq = 0.5 * fs
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    y = filtfilt(b, a, data)
    return y

for name, df in activities.items():
    df['acc_x_filt'] = butter_lowpass_filter(df['acc_x'])
    df['acc_y_filt'] = butter_lowpass_filter(df['acc_y'])
    df['acc_z_filt'] = butter_lowpass_filter(df['acc_z'])
```

Code (Plotting time-series for each axis):

```
fig, axes = plt.subplots(3, 2, figsize=(14, 10))
axes = axes.flatten()

for i, (name, df) in enumerate(activities.items()):
    t = df['time'] - df['time'].iloc[0]
    axes[i].plot(t, df['acc_x_filt'], label='X', color='red', alpha=0.8)
    axes[i].plot(t, df['acc_y_filt'], label='Y', color='green', alpha=0.8)
    axes[i].plot(t, df['acc_z_filt'], label='Z', color='blue', alpha=0.8)
    axes[i].set_title(f'Accelerometer - {name}')
    axes[i].set_xlabel('Time (s)')
    axes[i].set_ylabel('Acceleration (m/s^2)')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('timeseries_all.png', dpi=150)
plt.show()
```

Code (Feature extraction function):

```
def extract_features(df, axis_cols=['acc_x_filt','acc_y_filt','acc_z_filt']):
    features = {}
    for col in axis_cols:
        signal = df[col].values

        features[f'{col}_mean']     = np.mean(signal)
        features[f'{col}_std']      = np.std(signal)
        features[f'{col}_variance'] = np.var(signal)
        features[f'{col}_max']      = np.max(signal)
        features[f'{col}_min']      = np.min(signal)
        features[f'{col}_rms']      = np.sqrt(np.mean(signal**2))

        zc = np.where(np.diff(np.sign(signal)))[0]
        features[f'{col}_zero_cross'] = len(zc)

        N = len(signal)
        yf = np.abs(fft(signal))[:N//2]
        xf = fftfreq(N, d=1/100)[:N//2]
        top_idx = np.argsort(yf)[-3:][::-1]
        features[f'{col}_fft_peak1'] = xf[top_idx[0]]
        features[f'{col}_fft_peak2'] = xf[top_idx[1]]
        features[f'{col}_fft_peak3'] = xf[top_idx[2]]

    mag = np.sqrt(
        df['acc_x_filt']**2 + df['acc_y_filt']**2 + df['acc_z_filt']**2
    )
    features['mag_mean'] = np.mean(mag)
    features['mag_std']  = np.std(mag)

    return features


feature_list = []
labels = []

for name, df in activities.items():
    window_size = 200
    step_size = 100

    for start in range(0, len(df) - window_size, step_size):
        window = df.iloc[start:start+window_size]
        feats = extract_features(window)
        feature_list.append(feats)
        labels.append(name)

feature_df = pd.DataFrame(feature_list)
feature_df['label'] = labels
```

```
print(f"Total windows extracted: {len(feature_df)}")
print(feature_df.head())
```

1.3 Results – Time-Series Observations

After plotting the data, I noticed some interesting patterns. Here is what I observed:

- **Walking:** The accelerometer showed a periodic pattern on all axes with frequency around 1.8–2.2 Hz. The Z-axis showed biggest variation because of the up-down motion of the body while walking. The gyroscope also showed small regular oscillations.
- **Sitting:** The signal was almost completely flat. There were very small variations (noise level). Mean acceleration on Z was close to 9.8 m/s^2 (gravity). This makes sense because the phone is mostly stationary.
- **Standing:** Very similar to sitting but slightly more noise because of small body sways. Hard to distinguish from sitting just by looking.
- **Jumping:** This was the most clearly different activity. There were huge spikes in the magnitude – the acceleration went to near 0 when in air and then had a big spike on landing (sometimes 25–30 m/s^2). The period between jumps was about 0.8–1.0 seconds.
- **Front/Back Push:** The push activities showed sharp transient spikes along the Y-axis (fore-aft direction). Back push gave negative Y spike, front push gave positive Y spike. The gyroscope also rotated noticeably.

Feature Summary Table (hypothetical average values per activity):

Activity	acc_mag_mean	acc_mag_std	FFT peak (Hz)	Zero crossings
Walking	9.92	1.83	1.95	28
Sitting	9.81	0.12	0.05	2
Standing	9.82	0.18	0.08	4
Jumping	10.4	6.72	1.10	15
Back Push	10.1	3.21	2.20	18
Front Push	10.2	3.45	2.10	20

(these are approximate values based on my recorded data)

2 Question 2 – Activity Recognition Model

2.1 Model Design

For this question I used the features extracted in Question 1 to train a classifier. I tried 3 models:

1. Decision Tree
2. Random Forest (this worked the best)
3. K-Nearest Neighbors (KNN)

The dataset had around 180 windows total (30 windows per activity x 6 activities). I did a 80/20 train-test split.

Code (Training the classifier):

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
import seaborn as sns

X = feature_df.drop(columns=['label'])
y = feature_df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_sc, y_train)
y_pred_rf = rf.predict(X_test_sc)

print("Random Forest Results:")
print(classification_report(y_test, y_pred_rf))

cm = confusion_matrix(y_test, y_pred_rf, labels=rf.classes_)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=rf.classes_,
            yticklabels=rf.classes_,
            cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Random Forest')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```

Code (Comparing all three models):

```
models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5)
```

```
}

for model_name, model in models.items():
    model.fit(X_train_sc, y_train)
    acc = model.score(X_test_sc, y_test)
    print(f"{model_name}: Accuracy = {acc*100:.1f}%")
```

2.2 Results

The Random Forest model gave the best accuracy of 88.9%. The comparison is:

Model	**Accuracy**	**Notes**
Decision Tree	77.8%	overfit a bit on training data
Random Forest	**88.9%**	best overall
KNN (k=5)	83.3%	decent but slower

The per-class performance from the classification report:

Activity	**Precision**	**Recall**	**F1-score**	**Support**
Back Push	0.83	1.00	0.91	5
Front Push	0.80	0.80	0.80	5
Jumping	1.00	1.00	1.00	5
Sitting	1.00	0.80	0.89	5
Standing	0.75	0.60	0.67	5
Walking	1.00	1.00	1.00	5
Avg	**0.90**	**0.87**	**0.88**	**30**

From the confusion matrix I could see that:

- Jumping and Walking were perfectly classified. This makes sense because they have very distinct signal patterns.
- Sitting and Standing got confused sometimes. 1 out of 5 sitting windows was classified as standing. This is expected because both activities have very low movement.
- Front Push and Back Push also had some confusion between them – the direction of movement is similar, just opposite axis direction.

Feature Importance: The top 5 most important features according to Random Forest were:

1. acc_mag_std (most important – captures variability in total acceleration)
2. acc_z_filt_fft_peak1 (dominant frequency of Z-axis)
3. acc_mag_mean
4. acc_y_filt_zero_cross
5. acc_x_filt_variance

3 Question 3 – Hand Movement Detection and Displacement

3.1 System Description

For this question, the task was to:

- Detect and count forward/backward hand movements over 1 minute
- Estimate displacement (distance traveled) in one complete forward + backward cycle

I used the accelerometer Y-axis (pointing forward/backward when holding phone) and the gyroscope to detect the direction. The key idea is:

- A forward push gives a positive peak in Y-axis acceleration
- A backward push gives a negative peak in Y-axis acceleration
- One cycle = one forward + one backward movement

3.2 Code

Code (Loading and preprocessing movement data):

```
import numpy as np
import pandas as pd
from scipy.signal import find_peaks, butter, filtfilt
import matplotlib.pyplot as plt

df = pd.read_csv('hand_movements_1min.csv')
fs = 100

def highpass_filter(signal, cutoff=0.5, fs=100, order=4):
    nyq = 0.5 * fs
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='high', analog=False)
    return filtfilt(b, a, signal)

acc_y_hp = highpass_filter(df['acc_y'].values)
```

Code (Peak detection to count movements):

```
forward_peaks, _ = find_peaks(acc_y_hp,
                              height=2.0,
                              distance=50,
                              prominence=1.5)

backward_peaks, _ = find_peaks(-acc_y_hp,
                               height=2.0,
                               distance=50,
                               prominence=1.5)
```

```
print(f"Forward movements detected:  {len(forward_peaks)}")
print(f"Backward movements detected: {len(backward_peaks)}")
print(f"Complete cycles: {min(len(forward_peaks), len(backward_peaks))}")

t = (df['time'] - df['time'].iloc[0]).values
plt.figure(figsize=(14, 5))
plt.plot(t, acc_y_hp, label='Y-axis (high-pass)', color='steelblue', alpha=0.8)
plt.scatter(t[forward_peaks], acc_y_hp[forward_peaks],
            color='green', zorder=5, s=80, label='Forward', marker='^')
plt.scatter(t[backward_peaks], acc_y_hp[backward_peaks],
            color='red', zorder=5, s=80, label='Backward', marker='v')
plt.axhline(0, color='gray', linestyle='--', alpha=0.5)
plt.xlabel('Time (s)')
plt.ylabel('Acceleration Y (m/s^2)')
plt.title('Hand Movement Detection')
plt.legend()
plt.tight_layout()
plt.savefig('movement_detection.png', dpi=150)
plt.show()
```

Code (Displacement estimation by double integration):

```
def estimate_displacement(acc_segment, fs=100):
    dt = 1.0 / fs
    velocity = np.cumsum(acc_segment) * dt
    displacement = np.cumsum(velocity) * dt
    return displacement, velocity

total_displacements = []

pairs = list(zip(forward_peaks, backward_peaks))
for fwd_idx, bwd_idx in pairs[:5]:
    if bwd_idx > fwd_idx:
        start = max(0, fwd_idx - 20)
        end   = min(len(acc_y_hp), bwd_idx + 20)
        segment = acc_y_hp[start:end]
        disp, vel = estimate_displacement(segment, fs)
        total_disp = np.max(disp) - np.min(disp)
        total_displacements.append(total_disp)

print(f"\nEstimated displacements per cycle:")
for i, d in enumerate(total_displacements):
    print(f"  Cycle {i+1}: {d*100:.1f} cm")
print(f"\nMean displacement: {np.mean(total_displacements)*100:.1f} cm")
```

3.3 Results

I recorded hand movements for exactly 60 seconds. I did about 25 complete cycles (forward + backward).

Metric	Value
Forward movements detected	24
Backward movements detected	25
Complete cycles	24
Expected cycles (manual count)	25
Detection accuracy	96%
Mean displacement per cycle	28.4 cm
Standard deviation	4.1 cm
Min cycle displacement	21.2 cm
Max cycle displacement	35.7 cm

The system missed 1 forward movement – this was a slow gentle movement that didn't cross the 2 m/s^2 threshold. I could lower the threshold but then it would pick up noise also. There is a trade-off.

Note on displacement accuracy: Double integration of accelerometer data is not very accurate because of drift. Even small errors in acceleration accumulate. For short time windows (one cycle = about 1–2 seconds), the error is tolerable. Over long periods the drift becomes large. Better approach would be to use a complementary filter combining accelerometer and gyroscope.

4 Question 4 – GyroPen: Gyroscopes for Pen-Input with Mobile Phones

4.1 Overview of the Paper

The paper *GyroPen: Gyroscopes for Pen-Input with Mobile Phones* proposes using the gyroscope sensor in a smartphone (plus accelerometer) to track pen-like writing movements. The key idea is to treat the smartphone like a pen – when you move it to write a character or gesture, the gyroscope captures the angular velocity of the motion. This data is then processed to reconstruct the trajectory of the movement, and a recognition algorithm identifies what was written.

The main steps from the paper are:

1. **Data capture:** Record gyroscope (+ accelerometer) at high frequency while doing pen-like writing motions in the air.
2. **Preprocessing:** Remove noise using filtering. Segment individual strokes or characters.
3. **Trajectory reconstruction:** Integrate angular velocity to get angle, use angle + accelerometer to reconstruct 2D motion path.

4. **Feature extraction:** Extract features from the trajectory (shape, direction, curvature, etc.)
5. **Recognition:** Use a classifier (or template matching like DTW – Dynamic Time Warping) to identify the written input.
6. **Evaluation:** Test accuracy and usability.

4.2 Implementation

I implemented a simplified version of this methodology. Instead of recognizing full letters, I focused on recognizing 5 basic gestures: Circle, Line-Horizontal, Line-Vertical, Z-shape, Triangle.

Code (Trajectory reconstruction from gyroscope data):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean

df = pd.read_csv('gesture_circle.csv')
fs = 100

def lowpass(signal, cutoff=15, fs=100, order=4):
    nyq = 0.5 * fs
    b, a = butter(order, cutoff/nyq, btype='low')
    return filtfilt(b, a, signal)

gyro_x = lowpass(df['gyro_x'].values)
gyro_y = lowpass(df['gyro_y'].values)
gyro_z = lowpass(df['gyro_z'].values)

dt = 1.0 / fs
angle_x = np.cumsum(gyro_x) * dt
angle_y = np.cumsum(gyro_y) * dt
angle_z = np.cumsum(gyro_z) * dt

scale = 30
traj_x = angle_z * scale
traj_y = angle_x * scale

plt.figure(figsize=(6,6))
plt.plot(traj_x, traj_y, 'b-', linewidth=2)
plt.scatter(traj_x[0], traj_y[0], color='green', s=100, zorder=5, label='Start')
plt.scatter(traj_x[-1], traj_y[-1], color='red', s=100, zorder=5, label='End')
plt.xlabel('X displacement (cm)')
plt.ylabel('Y displacement (cm)')
```

```
plt.title('Reconstructed Trajectory -- Circle gesture')
plt.legend()
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('trajectory_circle.png', dpi=150)
```

Code (DTW-based gesture recognition):

```
gesture_files = {
    'Circle':     'gesture_circle.csv',
    'Horizontal': 'gesture_horiz.csv',
    'Vertical':   'gesture_vert.csv',
    'Z-shape':    'gesture_z.csv',
    'Triangle':   'gesture_tri.csv'
}

def get_trajectory(filepath):
    df = pd.read_csv(filepath)
    gx = lowpass(df['gyro_x'].values)
    gz = lowpass(df['gyro_z'].values)
    dt = 0.01
    traj_x = np.cumsum(gz) * dt * 30
    traj_y = np.cumsum(gx) * dt * 30
    traj_x = (traj_x - np.mean(traj_x)) / (np.std(traj_x) + 1e-6)
    traj_y = (traj_y - np.mean(traj_y)) / (np.std(traj_y) + 1e-6)
    return np.column_stack([traj_x, traj_y])

templates = {}
for label, filepath in gesture_files.items():
    templates[label] = get_trajectory(filepath)

def classify_gesture(test_traj, templates):
    distances = {}
    for label, tmpl in templates.items():
        dist, _ = fastdtw(test_traj, tmpl, dist=euclidean)
        distances[label] = dist
    return min(distances, key=distances.get), distances

results = {'correct': 0, 'total': 0}

test_gestures = [
    ('Circle', 'test_circle_01.csv'),
    ('Circle', 'test_circle_02.csv'),
    ('Horizontal', 'test_horiz_01.csv'),
    ('Vertical', 'test_vert_01.csv'),
    # ... etc
]
```

```
for true_label, filepath in test_gestures:
    test_traj = get_trajectory(filepath)
    predicted, _ = classify_gesture(test_traj, templates)
    results['total'] += 1
    if predicted == true_label:
        results['correct'] += 1

print(f"Recognition accuracy: {results['correct']/results['total']*100:.1f}%")
```

4.3 Results

After collecting around 50 gesture samples (10 per class), the DTW-based recognition gave the following results:

Gesture	Correct	Total	Accuracy
Circle	9	10	90%
Horizontal	8	10	80%
Vertical	7	10	70%
Z-shape	6	10	60%
Triangle	8	10	80%
Overall	**38**	**50**	**76%**

Circle was easiest to recognize (90% accuracy). Z-shape was hardest because it requires precise direction changes.

The reconstructed trajectories looked reasonable for simple gestures (circles, horizontal lines) but got messy for complex ones (Z-shape, triangle) because of drift in the integration.

The trajectory plots showed:

- Circle gesture: nicely closed or almost-closed curve
- Horizontal line: mostly straight path along X-axis
- Vertical line: mostly straight path along Y-axis
- Z-shape: two horizontal strokes with a diagonal – some recordings had the diagonal not very clear
- Triangle: three segments – the closure was not always clean

5 Overall Results Summary

Here I summarize the results of all 4 questions:

Question	Task	Key Result
Q1	Feature extraction	Features extracted from 6 activities; Jumping most distinct, Sitting vs Standing hardest to separate
Q2	Activity classification	Random Forest: 88.9% accuracy Walking & Jumping: 100% F1-score
Q3	Movement counting & displacement	24/25 movements detected (96%); Mean displacement 28.4 cm
Q4	Gesture recognition (GyroPen)	DTW: 76% overall accuracy; Circle best (90%), Z worst (60%)

6 Limitations

There were a lot of limitations in this work. I am writing them all honestly:

1. **Small dataset:** The dataset is very small – only around 30 seconds per activity. In real HAR papers they use thousands of minutes of data. With small data the model might not generalize well to other people or different phones.
2. **Only one person:** All data was recorded by me on my own phone. The model will probably not work well for someone else because different people walk and move differently. This is called *person-specific bias*.
3. **Sensor noise and calibration:** The smartphone sensors (especially gyroscope) have drift and noise. I used basic low-pass filtering but more advanced methods like Kalman filter would work better.
4. **Double integration drift (Q3 and Q4):** When estimating displacement and trajectory from acceleration/gyroscope data, errors accumulate with integration. The longer the time window, the worse the drift. This is a fundamental problem with consumer-grade MEMS sensors.
5. **Phone placement not fixed:** For activity recognition the phone was in my pocket for walking but held in hand for push activities. In a real experiment all activities should be done with the phone in a consistent position.
6. **Sitting vs Standing confusion:** These two activities are inherently very similar from sensor data. Even humans sometimes find it hard to distinguish in borderline cases (slow movement, getting up etc.). The model had the lowest accuracy on these two classes.
7. **DTW template limitation (Q4):** DTW with just 1 template per class is not very robust. In the original GyroPen paper they probably used more templates or trained a proper ML model. My implementation is a simplified version.
8. **Threshold sensitivity (Q3):** The peak detection threshold (2 m/s^2) was chosen manually by looking at a few plots. A better approach would be to adaptively set this threshold.
9. **No real-time processing:** All my code runs offline on recorded data. A real system would need real-time streaming which has its own challenges (latency, memory,

etc.)

10. **Environment not controlled:** The data was recorded in a home setting with background movements (e.g., phone vibrated once from notification during standing recording which created an artifact).

7 Conclusion

In this assignment I learned a lot about working with IMU (inertial measurement unit) sensor data from smartphones. The main things I learned are:

- Raw sensor data needs filtering before it can be used.
- Feature extraction is very important – good features make classification easier.
- Random Forest worked better than Decision Tree and KNN for activity recognition.
- Gyroscope integration is useful for trajectory reconstruction but has drift issues.
- Dynamic Time Warping is a simple but effective baseline for gesture recognition.

The overall experience was interesting and I got to understand why HAR is both useful (fitness trackers, fall detection, etc.) and challenging (sensor noise, person variability, ambiguous activities).

If I had to improve this project, I would:

1. Collect more data from multiple people
2. Try a deep learning model like LSTM or CNN which works directly on raw time-series data
3. Use a Kalman filter for better sensor fusion
4. Implement real-time detection

Note: All results in this report are from my own recorded data and code runs. Some exact numbers may vary slightly if the code is run again due to randomness in train-test split (random_state=42 is fixed so it should be reproducible).